

ADVANCED

Build a Sentiment Analysis Model

AI & ML Internship Track

Objective

Apply your skills to a real NLP (Natural Language Processing) problem: building a sentiment analysis model that classifies text (e.g. reviews or tweets) as positive or negative. This is your capstone task, combining data processing, modeling, and clear written explanation.

Task Requirements

- Use a public text dataset with sentiment labels (e.g. IMDB movie reviews, or a Twitter sentiment dataset from Kaggle).
- Preprocess the text data: cleaning, tokenization, and converting text into numerical features (e.g. using TF-IDF).
- Train a classification model on the processed text and evaluate it using accuracy and F1-score.
- Test your model on 3 of your own example sentences and show the predicted sentiment for each.
- Write a clear final summary explaining your full approach, results, and at least one limitation of your model.

Step-by-Step Guide

- 1 Find a sentiment-labeled text dataset — search Kaggle for 'sentiment analysis dataset' and pick one with clear positive/negative labels and at least a few thousand rows.
- 2 Install any missing libraries: `pip install scikit-learn nltk`.
- 3 Load your dataset and inspect the text and label columns. Check the balance of positive vs negative examples using `df['label'].value_counts()`.
- 4 Clean your text: lowercase everything, remove punctuation and special characters using Python's `re` module, e.g. `re.sub(r'^a-zA-Z\s|', '', text)`.
- 5 Convert your cleaned text into numeric features using TF-IDF: `from sklearn.feature_extraction.text import TfidfVectorizer, then vectorizer =`

```
TfidfVectorizer(max_features=5000) and X =  
vectorizer.fit_transform(df['cleaned_text']).
```

- 6 Split into train/test sets as in previous weeks, and train a classification model — Logistic Regression or Multinomial Naive Bayes both work well for text classification.
- 7 Evaluate using accuracy and F1-score as in Week 2's task, and consider printing a confusion matrix using `from sklearn.metrics import confusion_matrix` for extra insight.
- 8 Write 3 of your own example sentences (e.g. 'This was the best experience ever' and 'This was a complete waste of time'), transform them using the same vectorizer (`vectorizer.transform([...])`), and predict their sentiment using your trained model.
- 9 In a final markdown section, summarize your full pipeline from raw text to prediction, your model's performance, and one honest limitation (e.g. struggles with sarcasm, small dataset size, etc.).
- 10 Push your completed notebook to GitHub as instructed below — this is your final task, so make sure your README clearly explains the whole project.

Submission Instructions

1. Push your completed work to a public GitHub repository.
2. Name your repository clearly, e.g. *vortextech-aiml-week4*.
3. Make sure your repository includes a short README explaining what you built and how to run it.
4. Fill out the Week 4 submission form (link provided via email) with your name, track, and GitHub repository link before the deadline.

Estimated Time

12-15 hours — this is the most advanced AI/ML task in the program, allow extra time for text preprocessing.